



Course Code	CSA03	Course Title	Data Structures
CO Assessed	CO1	Assessment	Assessment Tool 2: Technical Presentation
Weightage	30%	Marks	10 (Question 32)
CO1 Statement	<i>Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)</i>		
Student Name	sk.md.suhail	Reg. No.	192525229
Section/Batch	Slot D	Date	05-10-2026

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

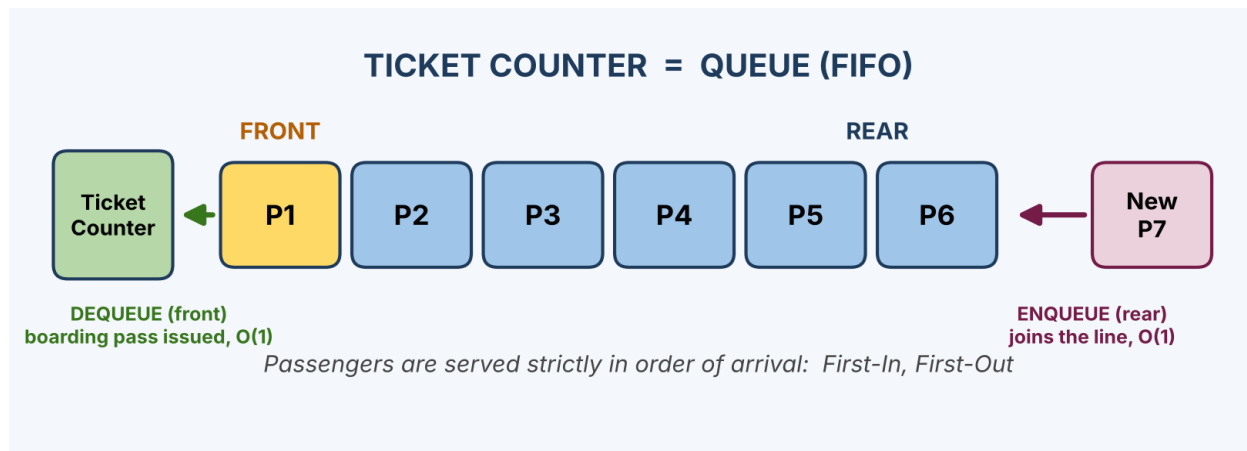
Code of Conduct:

I sk.md.suhail (192525229) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: sk.md.suhail

Q32. A ticket counter processes passengers waiting in line for boarding passes. Suggest an efficient data structure and explain why it is appropriate.

TECHNICAL PRESENTATION



1. Problem Identification

At a ticket counter, passengers arrive one after another and wait in a line. Each passenger must be served **strictly in the order of arrival** – the person who came first gets the boarding pass first, and a new passenger always joins at the back. The system therefore needs only two operations: insert at one end and remove from the other. Searching or reordering the line is not required.

2. Suggested Data Structure: Queue (FIFO)

The suitable data structure is a **Queue**, a linear ADT that follows the **First-In, First-Out (FIFO)** principle. Two pointers are maintained: **front** (the passenger being served next) and **rear** (where a new passenger joins). It can be implemented using a circular array or a linked list with front and rear pointers.

3. Operations and Complexity Analysis

Operation	Action at the ticket counter	Time	Space
enqueue(x)	New passenger joins at the rear of the line	O(1)	O(1)
dequeue()	Passenger at the front gets the boarding pass and leaves	O(1)	O(1)
peek() / front()	See who is next to be served	O(1)	O(1)
isEmpty() / isFull()	Check whether anyone is waiting / line is at capacity	O(1)	O(1)
Whole line of n passengers	Store all waiting passengers	—	O(n)

Every operation only moves a pointer, so none of them depends on how many passengers are waiting. Time complexity is **O(1)** for both enqueue and dequeue, and the space required is **O(n)** for n waiting passengers.

4. Comparison with Other Data Structures

Data structure	Serve next	Add new	Serves in arrival order?	Verdict
Queue (circular array / linked list with front & rear)	O(1)	O(1)	Yes (FIFO)	Best choice
Array with shifting after each removal	O(n)	O(1)	Yes	Slow – shifts n elements
Singly linked list with head pointer only	O(1)	O(n)	Yes	Slow – walks to the tail
Stack	O(1)	O(1)	No (LIFO)	Unfair – last person served first
Priority queue (heap)	O(log n)	O(log n)	No (by priority)	Unnecessary overhead

5. Why the Queue Is Appropriate

- **Fairness:** FIFO guarantees that no passenger is skipped or served out of turn; this matches the real-life line exactly.
- **Efficiency:** Joining and leaving the line both take constant time, so the counter stays fast even when hundreds of passengers are waiting.
- **Simplicity:** Only two pointers (front and rear) are needed, which keeps the code small and easy to verify.
- **Memory use:** A circular array reuses the slots freed by served passengers, so memory stays O(n) and no shifting is needed.
- **Natural fit for the ADT:** The problem is exactly the Queue ADT – insertion at the rear and deletion at the front – with no extra features paid for.

6. Implementation Sketch (Circular Queue in C)

```
#define MAX 100
int q[MAX], front = 0, rear = -1, count = 0;

void enqueue(int id) { // O(1)
    if (count == MAX) { printf("Line full\n"); return; }
    rear = (rear + 1) % MAX; // circular: reuse freed slots
    q[rear] = id; count++;
}

int dequeue() { // O(1)
    if (count == 0) { printf("No one waiting\n"); return -1; }
    int id = q[front];
    front = (front + 1) % MAX; count--;
    return id; // passenger gets boarding pass
}
```

7. Conclusion

A ticket counter serving passengers in the order they arrive is best modelled by a **Queue**. It serves passengers fairly (FIFO), performs enqueue and dequeue in O(1) time, uses O(n) space, and is simpler and faster than arrays with shifting, stacks or priority queues for this purpose.